

Research Document

1. Screen-Shooting Resistant Watermarking With Grayscale Deviation Simulation

Source details: Yiyi Li, Xin Liao, and Xiaoshuai Wu. IEEE Transactions on Multimedia, 2024. DOI: 10.1109/TMM.2024.3415415.

Link: <https://ieeexplore.ieee.org/document/10559271/>

This is one of the most important sources because it deals with a common way confidential information leaks: somebody takes a screenshot or captures the screen while a document is open. A normal hidden marker may disappear when text is converted into an image, resized, or compressed. This research studies watermarking methods that are designed to remain detectable after that type of screen-capture distortion. It supports the project's live demonstration where a recipient's marked document is screenshotted, compressed, and then examined to recover the hidden identifier. In simple words, it helps prove that the project is not only marking a file, but is trying to make the mark survive a realistic leakage method.

How this supports the proposed solution: The project should embed a short secret event token into the rendered visual document, such as a PDF page or controlled viewer output. The leaked screenshot can then be passed to a watermark extractor, which returns the token and uses it to find the matching signed record in the ledger.

2. Universal Screen-Shooting Robust Image Watermarking With Channel-Attention in DCT Domain

Source details: Fang Cao, Daidou Guo, Tianjun Wang, Heng Yao, Jian Li, and Chuan Qin. Expert Systems with Applications, 2024. DOI: 10.1016/j.eswa.2023.122062.

Link: <https://www.sciencedirect.com/science/article/abs/pii/S0957417423025642>

This source gives a second, independent research basis for watermarking that can survive screen capture. It is useful because a screenshot turns a document into pixels; therefore, a hidden marker that exists only as invisible text characters or metadata may not survive. The paper focuses on embedding information in a way that is difficult for a person to notice but can still be extracted after visual changes. This directly supports the requirement that every recipient's copy should look normal while secretly carrying a unique identifier. It also supports testing the file after JPEG recompression, resizing, and similar changes that commonly happen when files are shared.

How this supports the proposed solution: The prototype should use a controlled document-rendering path, such as text converted into a watermarked PDF or a dedicated local viewer. The team should measure how often the hidden token can still be extracted after screenshots, resizing, JPEG compression, and small crops.

3. FontCode: Embedding Information in Text Documents Using Glyph Perturbation

Source details: Chengzheng Yu et al. ACM Symposium on User Interface Software and Technology, 2018. DOI: 10.1145/3152823.

Link: <https://dl.acm.org/doi/abs/10.1145/3152823>

It is especially valuable because it focuses on text documents rather than only generic images. It shows that tiny changes to the shapes of letters can carry hidden machine-readable information while the page still appears normal to a reader. That idea is very close to the goal of giving each recipient a visibly identical but secretly different document. The paper also makes the project easier to understand: the watermark does not have to be a visible name printed across the page; it can be a subtle pattern in the document's rendering. It is especially relevant to future work involving OCR, printed pages, and scanned copies.

How this supports the proposed solution: For a controlled-font demonstration, the team can embed an opaque random code rather than a person's name directly into the text rendering. The system then keeps the actual identity, document hash, timestamp, and signature safely in the ledger and only uses the recovered code to look up that evidence.

4. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)

Source details: National Institute of Standards and Technology, finalized August 2024. NIST FIPS 203.

Link: <https://csrc.nist.gov/pubs/fips/203/final>

This is an official NIST standard rather than an ordinary research paper, which makes it a very strong source for the encryption part of the project. The project requires post-quantum cryptography, meaning cryptography selected to resist future quantum-computer attacks. ML-KEM is used to securely establish or protect the secret key material that lets one approved recipient access a protected document. It supports the "encrypt once, give access separately to each recipient" design. This is important because the system should not keep a plain document available for anyone to copy before the watermarking and logging process happens.

How this supports the proposed solution: The correct build pattern is to create one random symmetric key for the document, encrypt the document once with that key, and then use ML-KEM separately for every recipient to protect access to that document key. This gives every approved recipient their own cryptographic access path without repeatedly encrypting the entire large file.

5. FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)

Source details: National Institute of Standards and Technology, finalized August 2024. NIST FIPS 204.

Link: <https://csrc.nist.gov/pubs/fips/204/final>

This official NIST standard supports the part of that says the recipient should sign the decryption record using their own private key. A hidden watermark alone can be questioned: someone may say it was inserted later or that a database was edited. A digital signature creates a checkable cryptographic link between a person's enrolled signing key and the exact record they signed. ML-DSA is the project's post-quantum signature algorithm, so it supports both record integrity and a strong evidence trail for future verification. It turns the system from "we found a hidden code" into "we found a hidden code that matches a signed and unchanged event record."

How this supports the proposed solution: After watermark creation, the recipient device should sign a standard record containing the document version hash, a random event ID, the hidden-token commitment, the recipient certificate ID, the software version, the time recorded by the system, and the hash of the previous ledger record. Later, anyone with the recipient's public key can verify that this exact record was signed and was not changed afterward.

6. Blockchain-Powered Forensic Evidence Storage: A New Approach to Data Security

Source details: 2024 IEEE Pune Section International Conference (PuneCon). IEEE Xplore document 10895214.

Link: <https://ieeexplore.ieee.org/document/10895214/>

This paper supports the evidence-storage part of the system. The project correctly recognizes that an ordinary access-log database is not enough if a powerful administrator can edit or delete records. Forensic evidence needs a history that makes hidden changes detectable. This source discusses blockchain-style storage for preserving traceability,

authenticity, and the integrity of evidence. It supports the idea of preserving signed decryption records in an append-only history instead of storing them as ordinary editable log entries.

How this supports the proposed solution: The prototype should link each new record to the previous one using a hash, so changing an older record breaks the chain. It should also require independent approvals, for example two out of three ledger custodians, before treating a ledger checkpoint as final. This prevents one administrator from silently rewriting history if the custodians and keys are genuinely separate.